

STM32 Firmware Bring-Up Using VS Code

Cross Compilation • Startup File • Linker Script • Flashing • Debugging

Purpose: This guide explains what changes when you move from STM32CubeIDE to a VS Code based workflow. You will explicitly manage the ARM cross compiler, startup file, linker script, build system, flashing and debugging setup. The goal is to make your STM32 board boot your own firmware safely and professionally.

One-line answer

Yes. When you use VS Code instead of STM32CubeIDE, your firmware must be cross-compiled for ARM Cortex-M, linked using a linker script, converted to .bin/.hex, flashed through ST-LINK, and debugged using GDB/OpenOCD or ST-LINK GDB Server. Your board will boot using the startup file and vector table included in your firmware.

1. What you are actually doing

You are not permanently changing or damaging the STM32 board. You are replacing the application firmware stored in internal Flash. After reset, the MCU starts from the vector table in Flash and runs your Reset_Handler before reaching main().

```
Power ON / Reset
-> CPU reads vector table from Flash at 0x08000000
-> Loads initial stack pointer
-> Jumps to Reset_Handler
-> Startup code initializes RAM sections
-> Optional SystemInit / clock init
-> Calls main()
-> Your BSP and application start
```

2. Why cross compilation is required

Your laptop/desktop runs on x86/x64, but STM32 runs ARM Cortex-M machine code. Normal gcc builds programs for your PC. For STM32, you need arm-none-eabi-gcc.

Compiler	Target	Example output
gcc	PC / laptop	Windows/Linux executable
arm-none-eabi-gcc	Bare-metal ARM Cortex-M	firmware.elf / firmware.bin / firmware.hex

```
For STM32F407 / Cortex-M4:
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard
```

```
For STM32H743 / Cortex-M7:
arm-none-eabi-gcc -mcpu=cortex-m7 -mthumb -mfpu=fpv5-d16 -mfloat-abi=hard
```

3. Minimum project files

A from-scratch STM32 firmware project needs these core files. You can initially use ST-provided startup and linker files, then gradually customize them.

File	Purpose	Beginner advice
startup_stm32xxxx.s	Contains vector table, Reset_Handler and default interrupt handlers	Start with ST-provided file, then study it
linker_script.ld	Places code/data into Flash and RAM	Start with board-specific ST linker script
main.c	Your application entry point	Keep first version simple: LED blink
system_stm32xxxx.c or bsp_clock.c	Clock and system initialization	Use simple internal clock first, PLL later
CMSIS/device headers	Register definitions and core access	Use CMSIS; do not reinvent register addresses initially
CMakeLists.txt or Makefile	Build instructions	Start with Makefile or simple CMake, then improve

4. Startup file: what it does

The startup file is the first firmware code path after reset. It prepares the C runtime environment and then calls main().

```
Reset_Handler concept:
  Copy initialized variables: .data from Flash to RAM
  Clear uninitialized variables: .bss to zero
  Call SystemInit() or BSP_Clock_Init()
  Call main()
```

```
Vector table concept:
  Initial stack pointer
  Reset_Handler
  NMI_Handler
  HardFault_Handler
  SysTick_Handler
  USARTx_IRQHandler
  SPIx_IRQHandler
  EXTIx_IRQHandler
```

5. Linker script: what it does

The linker script tells the linker where Flash and RAM start, their sizes, and where to place each program section.

```
Typical STM32 memory idea:
FLASH starts at 0x08000000
SRAM  starts at 0x20000000

.text  -> code and constants, stored in Flash
.data  -> initialized global/static variables, loaded from Flash and copied to RAM
.bss   -> zero-initialized global/static variables, placed in RAM
.stack -> runtime call stack in RAM
.heap  -> dynamic allocation area, if used
```

6. Build outputs you will generate

Output	Meaning
.elf	Main firmware image with debug symbols. Used for debugging.
.bin	Raw binary image. Often used for flashing at 0x08000000.
.hex	Intel HEX format. Also commonly used for flashing.
.map	Memory map showing section placement, symbol sizes and Flash/RAM usage.

7. Toolchain setup with VS Code

Tool	Role
VS Code	Code editor and task launcher
Arm GNU Toolchain	Cross compiler: arm-none-eabi-gcc, objcopy, size, gdb
CMake + Ninja	Professional build system
STM32CubeProgrammer CLI	Simple flashing through ST-LINK
OpenOCD or ST-LINK GDB Server	Debug server for GDB
Cortex-Debug extension	VS Code debug frontend for Cortex-M

8. Recommended folder structure

```
/firmware
/startup
  startup_stm32f407xx.s
/linker
  STM32F407VGTX_FLASH.ld
/cmsis
/bsp
  bsp_clock.c
  bsp_gpio.c
  bsp_uart.c
/drivers
  spi_ll.c
  i2c_ll.c
  rfid_mfrc522.c
/app
  app_main.c
  basket_manager.c
/cmake
  arm-none-eabi.cmake
CMakeLists.txt
```

9. Typical CMake build flow

```
cmake -S . -B build -G Ninja -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake
cmake --build build
arm-none-eabi-size build/firmware.elf
arm-none-eabi-objcopy -O binary build/firmware.elf build/firmware.bin
arm-none-eabi-objcopy -O ihex build/firmware.elf build/firmware.hex
```

10. Flashing the firmware

After the build, flash the binary or ELF to STM32 internal Flash through ST-LINK.

```
Using STM32CubeProgrammer CLI:
STM32_Programmer_CLI -c port=SWD -w build/firmware.bin 0x08000000 -v -rst

Using OpenOCD:
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \
-c "program build/firmware.elf verify reset exit"
```

11. Debugging with VS Code

Debugging requires arm-none-eabi-gdb plus a debug server such as OpenOCD or ST-LINK GDB Server. The VS Code Cortex-Debug extension gives you breakpoints, stepping, variable watch, register view and memory view.

```
VS Code Cortex-Debug flow:
1. Build firmware.elf
2. Start OpenOCD / ST-LINK GDB Server
3. arm-none-eabi-gdb connects to target
4. Load symbols from firmware.elf
5. Set breakpoints and step through Reset_Handler, main(), BSP init and drivers
```

12. Safe beginner bring-up sequence

Step	Goal	Success proof	Common failure
1	Use ST startup + linker	Project builds to .elf	Missing include paths or wrong CPU flags
2	Minimal main()	Debugger reaches main	Bad vector table or linker script
3	GPIO LED blink	LED toggles	Wrong GPIO port/pin or clock not enabled
4	UART printf	Serial terminal shows text	Wrong baud rate, wrong alternate function
5	SysTick/timer delay	Accurate periodic blink	Wrong system clock value
6	SPI/I2C basic test	Read WHO_AM_I / RFID version register	Pin alternate function or bus timing issue
7	Add application layer	Clean BSP-driver-app separation	Mixing board mapping into app code

13. BSP vs low-level driver in this setup

This distinction is important when creating a clean project architecture.

Layer	Knows	Example
BSP	Board-specific wiring and initialization	RFID uses SPI1, CS is PA4, ESP32 uses USART2
MCU low-level driver	STM32 peripheral registers	SPI_TransferByte(), UART_Send(), GPIO_WritePin()
Device driver	External device protocol	RFID_ReadUID(), OLED_Print(), ESP32_SendAT()
Application	Product logic	Basket_AddItem(), Calculate_Total(), Generate_Checkout()

14. Recovery and safety notes

- A bad firmware image usually causes HardFault, no UART output, no LED blink, or debugger connection issues. It normally does not damage the board.
- You can erase and reflash using STM32CubeProgrammer, OpenOCD, STM32CubeIDE, or another known-good project.
- Start with ST-provided startup/linker files and customize gradually.
- Keep BOOT0/boot mode knowledge handy for recovery if the application locks the MCU very early.
- Do not enable complex PLL, cache, MPU, DMA or RTOS before the basic LED and UART bring-up is stable.

15. Practical milestone checklist

- ☐ Toolchain installed and arm-none-eabi-gcc visible in terminal
- ☐ VS Code project builds without STM32CubeIDE
- ☐ firmware.elf, firmware.bin, firmware.hex and firmware.map generated
- ☐ Board flashed successfully through ST-LINK
- ☐ Debugger reaches Reset_Handler and main
- ☐ LED blink works from register-level GPIO
- ☐ UART debug print works
- ☐ BSP files created: clock, gpio, uart, timer
- ☐ Low-level peripheral driver separated from board pin mapping
- ☐ README documents build, flash and debug commands

16. Final mental model

```
PC source code
-> CMake / Ninja
-> arm-none-eabi-gcc cross compiler
-> linker script places code/data into Flash/RAM
-> firmware.elf with symbols
-> firmware.bin / firmware.hex
-> ST-LINK flashes STM32 internal Flash
-> CPU resets and reads vector table
-> startup file runs Reset_Handler
-> main() starts
-> BSP initializes board
-> drivers operate peripherals
-> application runs
```